# Validation expérimentale de la détection d’attaques web par Wazuh

Déploiement d’une application volontairement vulnérable, instrumentation des journaux et évaluation par attaques contrôlées

**Nature du document :** rapport académique

**Plateforme étudiée :** DVWA, Apache, PHP, MySQL, Wazuh

**Date :** 03/08/2026

## Résumé

Ce rapport présente la mise en œuvre d'un environnement expérimental destiné à vérifier la capacité de Wazuh à superviser et à signaler des activités offensives visant une application web. L'expérimentation repose sur l'installation de l'application Damn Vulnerable Web Application (DVWA) sur une machine agent Ubuntu, l'activation de la collecte des journaux associés au service Apache, puis la conduite d'attaques contrôlées depuis une machine Kali Linux.

Les scénarios menés couvrent une phase de reconnaissance, une tentative de force brute sur le mécanisme d'authentification et une exploitation par injection SQL, d'abord manuelle puis automatisée. Les résultats montrent que les événements générés par ces actions sont correctement observables depuis l'interface du manager Wazuh, ce qui permet de confirmer la cohérence de la chaîne de détection mise en place.


## 1. Introduction

La supervision des applications web constitue un enjeu central en cybersécurité, car les services HTTP exposés représentent une surface d'attaque particulièrement fréquente. Dans ce contexte, l'objectif du travail réalisé a été de construire un laboratoire réaliste mais contrôlé permettant d'observer la génération de traces liées à des comportements offensifs, puis de vérifier la capacité d'un système de détection basé sur Wazuh à centraliser et à rendre visibles ces événements.

L'approche retenue consiste à déployer une application volontairement vulnérable sur une machine supervisée, à provoquer des actions malveillantes depuis une station d'attaque dédiée, puis à étudier la remontée des journaux et des alertes correspondants dans l'interface du manager. Une telle démarche permet de rapprocher l'analyse théorique des mécanismes de détection d'une validation concrète fondée sur l'observation de données réelles issues du système testé.

## 2. Objectifs de l'expérimentation

Les objectifs poursuivis dans cette expérimentation sont les suivants :

1. installer un serveur web vulnérable sur la machine agent afin de disposer d'une cible représentative ;
2. configurer la collecte et la centralisation des journaux applicatifs et web au sein de Wazuh ;

3. exécuter plusieurs scénarios d’attaque contrôlée depuis Kali Linux ;
4. vérifier que les actions menées génèrent des événements observables et exploitables sur le manager.

# 3. Environnement expérimental

L’architecture du laboratoire repose sur trois éléments complémentaires : une machine Ubuntu configurée en tant qu’agent Wazuh et hébergeant l’application vulnérable, une machine Kali Linux utilisée comme poste d’attaque, et un manager Wazuh couplé à OpenSearch Dashboards pour la visualisation et l’analyse des journaux centralisés.

| Élément | Fonction dans l’expérimentation |
|---|---|
| Agent Ubuntu | Exécute Apache, PHP, MySQL et DVWA ; produit les journaux techniques et applicatifs surveillés. |
| Kali Linux | Permet l’exécution des scénarios de reconnaissance, de force brute et d’injection SQL. |
| Manager Wazuh / Dashboards | Centralise, indexe et rend consultables les événements remontés depuis l’agent. |

# 4. Déploiement de l’application vulnérable

## 4.1 Installation de la pile applicative

La première étape a consisté à installer les composants nécessaires à l’exécution de DVWA, à savoir le serveur HTTP Apache, le moteur de base de données MySQL, l’interpréteur PHP ainsi que plusieurs dépendances utiles au bon fonctionnement de l’application. Cette phase établit la base technique du laboratoire de test.

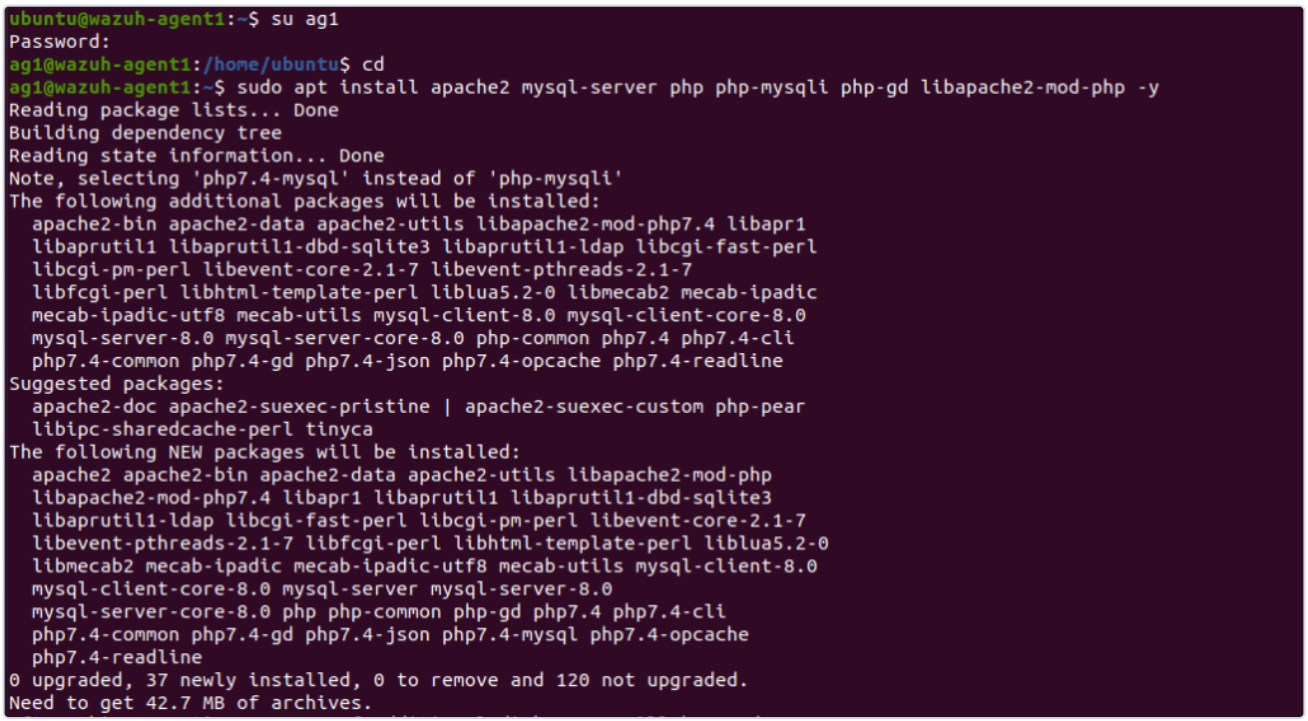



Figure 1 — Installation de la pile logicielle Apache, MySQL et PHP sur la machine agent.

## 4.2 Initialisation de la base de données de l’application

Afin d’assurer le fonctionnement de DVWA, une base de données dédiée a été créée, de même qu’un compte local disposant des droits requis. Cette configuration permet à l’application web d’interagir avec le moteur SQL dans des conditions suffisamment proches d’un contexte réel d’exploitation.

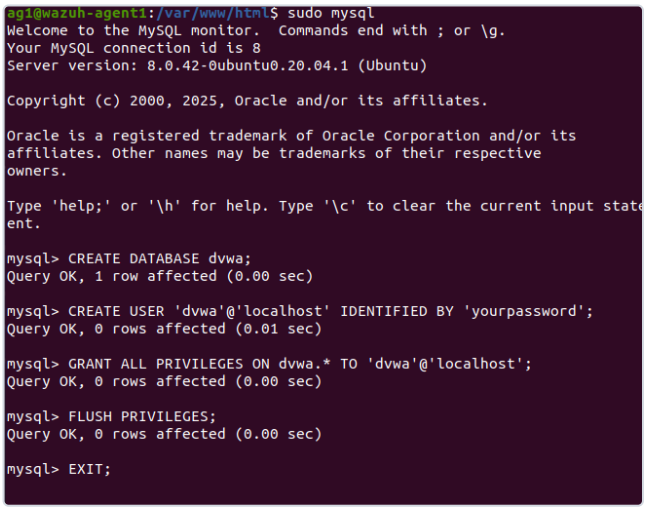



Figure 2 — Création de la base *dvwa* et de l’utilisateur associé dans MySQL.

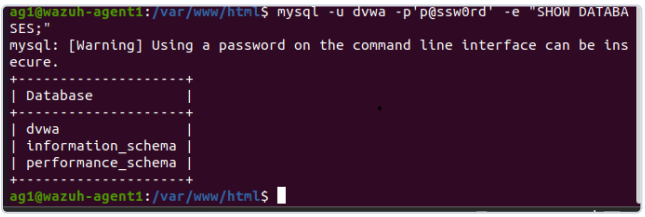



Figure 3 — Vérification en ligne de commande de l’existence de la base *dvwa*.

### 4.3 Mise à disposition de l’interface DVWA

Après configuration du service web, l’application a été rendue accessible depuis un navigateur. Les captures ci-dessous montrent d’une part les indications relatives à l’initialisation et à la connexion, et d’autre part la page d’authentification effectivement présentée à l’utilisateur.




Figure 4 — Accès à la page d’initialisation et à la page de connexion de DVWA.

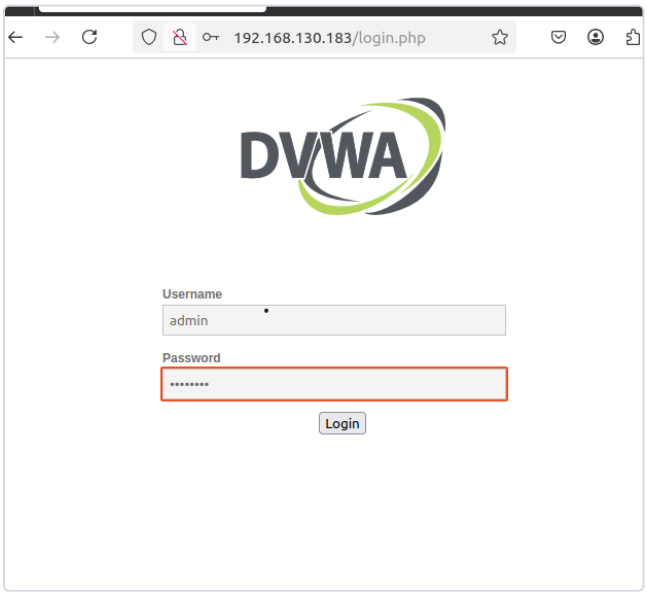



Figure 5 — Interface de connexion de l’application DVWA.

## 5. Instrumentation et visibilité dans Wazuh

Une fois le service web opérationnel, la collecte des événements a été vérifiée dans l’environnement Wazuh. La présence de l’index `wazuh-archives-*` confirme que les traces archivées sont bien reçues et indexées. La consultation d’événements liés à `login.php` montre par ailleurs que les accès web générés par les interactions avec l’application sont effectivement observables au niveau du manager.

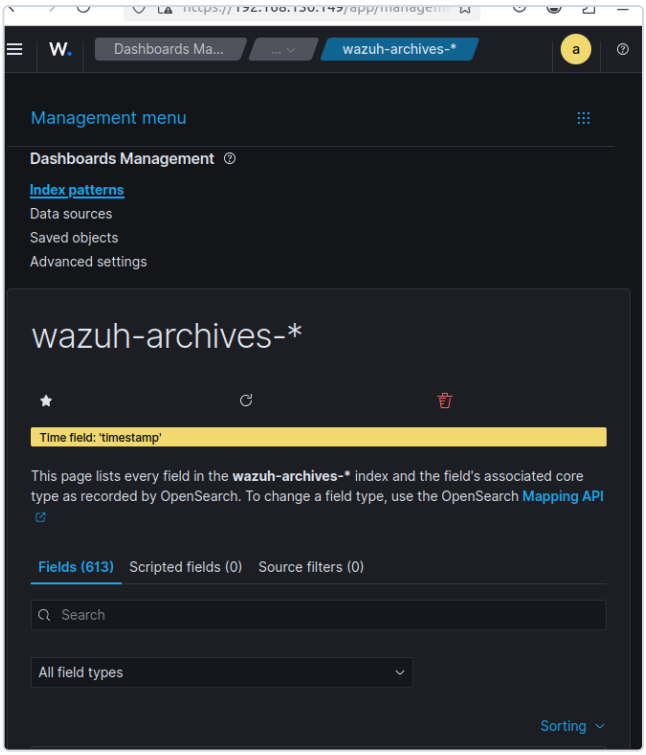



Figure 6 — Présence et configuration de l'index *wazuh-archives-** dans Dashboards.

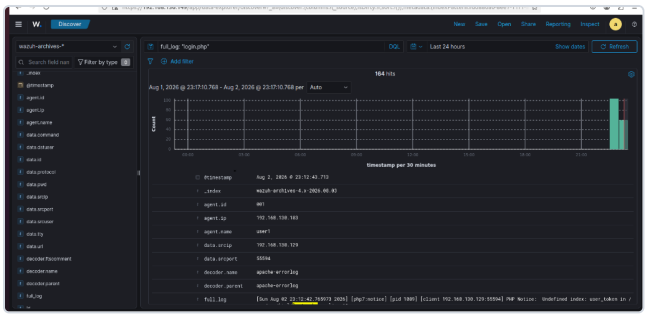


Figure 7 — Visualisation des événements concernant *login.php* dans l'interface Discover.

Cette étape est déterminante, car elle valide la disponibilité du socle d'observation nécessaire à la suite de l'expérimentation. En l'absence de centralisation correcte des journaux, il serait impossible d'interpréter les effets des attaques menées depuis la machine Kali.

# 6. Méthodologie d'évaluation par attaques contrôlées

L'évaluation repose sur une logique progressive. Une première phase de reconnaissance vise à cartographier partiellement les ressources exposées par le serveur. Une seconde phase s'intéresse au mécanisme d'authentification à travers une tentative de force brute. Enfin, une troisième phase porte sur l'exploitation d'une vulnérabilité d'injection SQL, d'abord validée manuellement, puis confirmée par un outil d'automatisation spécialisé.

## 6.1 Reconnaissance de la surface web

L'outil Gobuster a été employé afin d'énumérer les répertoires et certaines ressources accessibles sur le serveur. Les résultats font apparaître plusieurs chemins intéressants, notamment des répertoires de configuration, de documentation et des points d'entrée applicatifs. Cette phase illustre la facilité avec laquelle un attaquant peut acquérir une première compréhension de l'exposition du service.

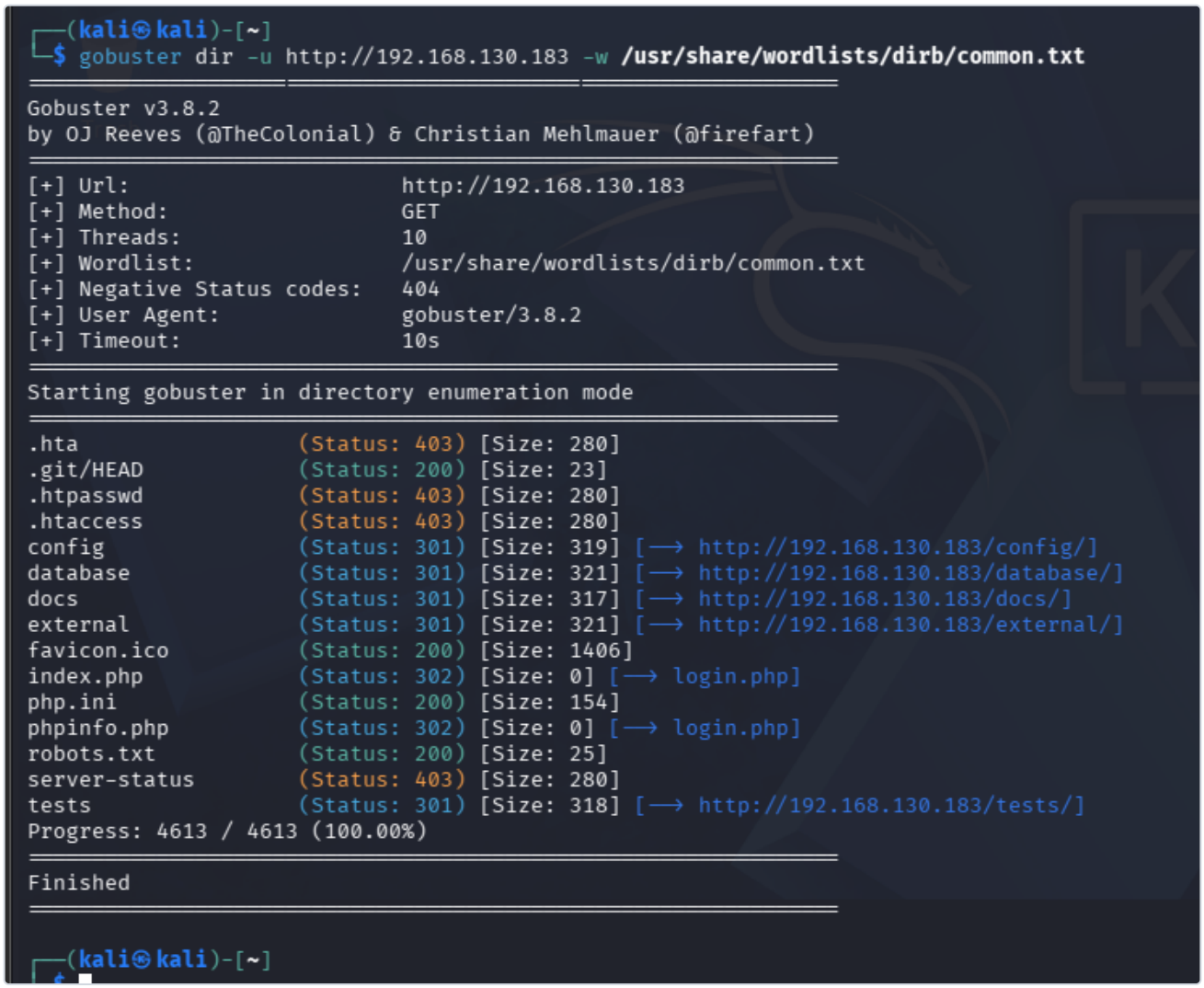



Figure 8 — Énumération de répertoires et de ressources web à l'aide de Gobuster.

## 6.2 Tentative de force brute sur l'authentification

Une attaque de type force brute a été dirigée vers le formulaire d'authentification de DVWA à l'aide d'Hydra. La capture fournie montre que, dans ce test précis, aucune paire identifiant/mot de passe valide n'a été trouvée. Néanmoins, d'un point de vue défensif, cette phase demeure pertinente, car elle provoque une répétition de requêtes d'authentification susceptibles de laisser une signature claire dans les journaux.

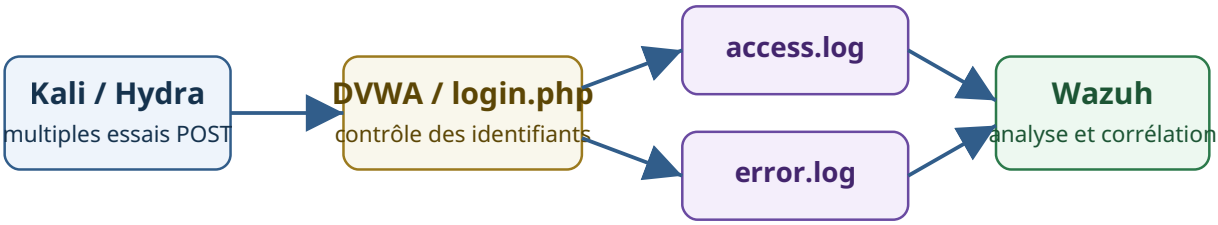



Schéma 1 — Représentation simplifiée du déroulement d'une tentative de force brute sur le formulaire d'authentification.

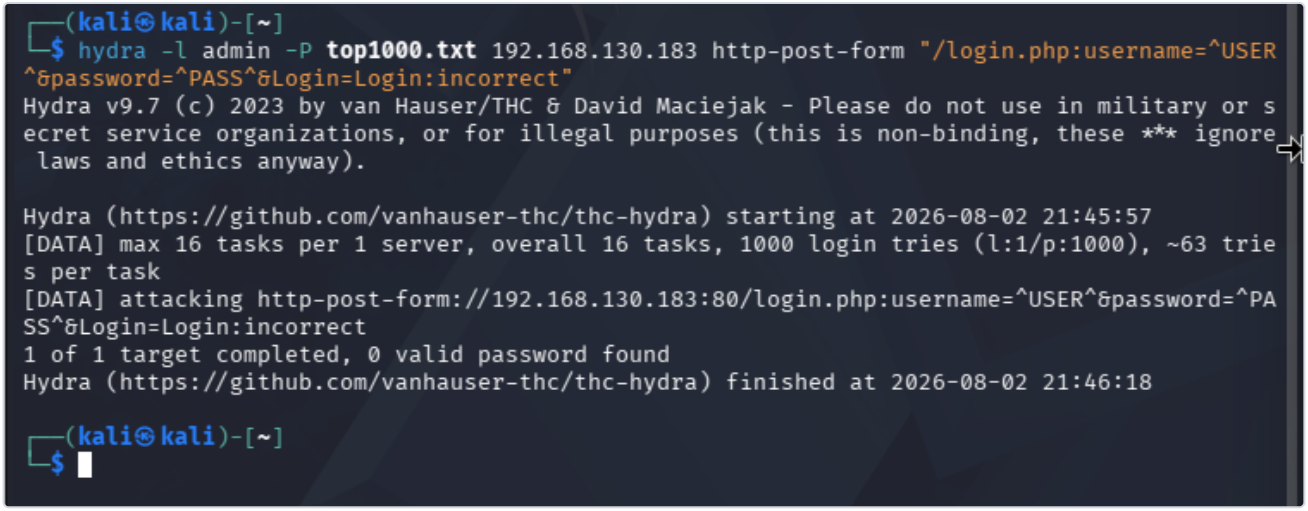



Figure 9 — Exécution d'une tentative de force brute sur *login.php* à l'aide d'Hydra.

## 6.3 Exploitation par injection SQL

La partie la plus significative de l'expérimentation a porté sur l'injection SQL dans le module vulnérable de DVWA. Une préparation a d'abord été effectuée afin de manipuler correctement les jetons de session et de sécurité. Une requête contenant une charge utile adaptée a ensuite été envoyée sur le paramètre `id`, ce qui a permis de confirmer un comportement anormal révélateur d'une concaténation non sécurisée des données d'entrée dans la requête SQL sous-jacente.

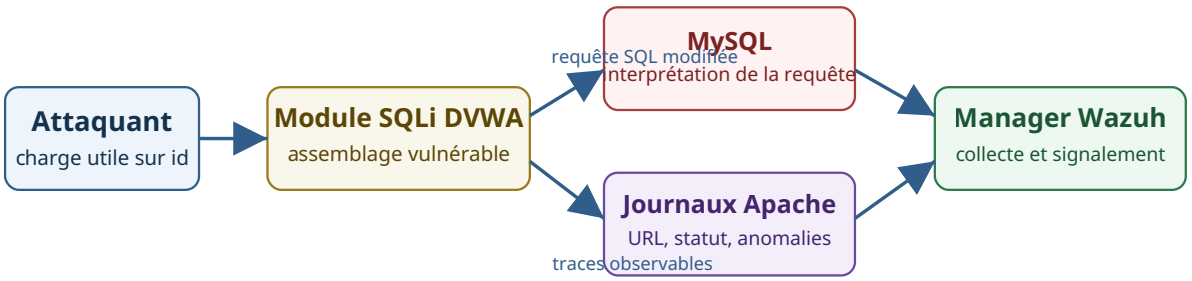



Schéma 2 — Représentation simplifiée d'une exploitation par injection SQL et de sa visibilité dans Wazuh.

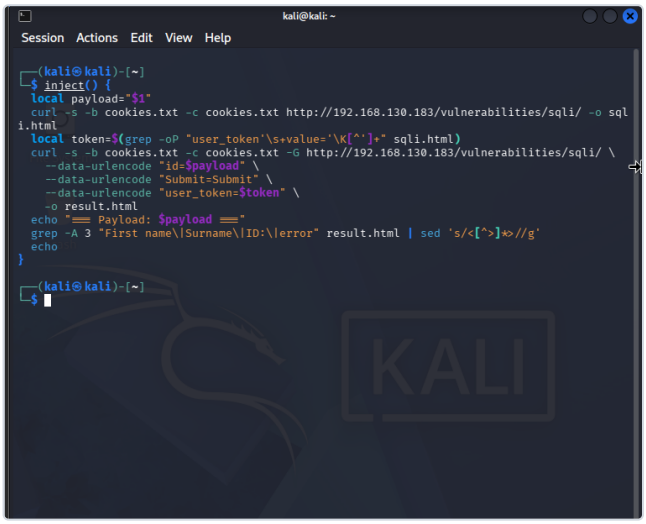



Figure 10 — Préparation d'une fonction shell facilitant la répétition des requêtes d'injection.

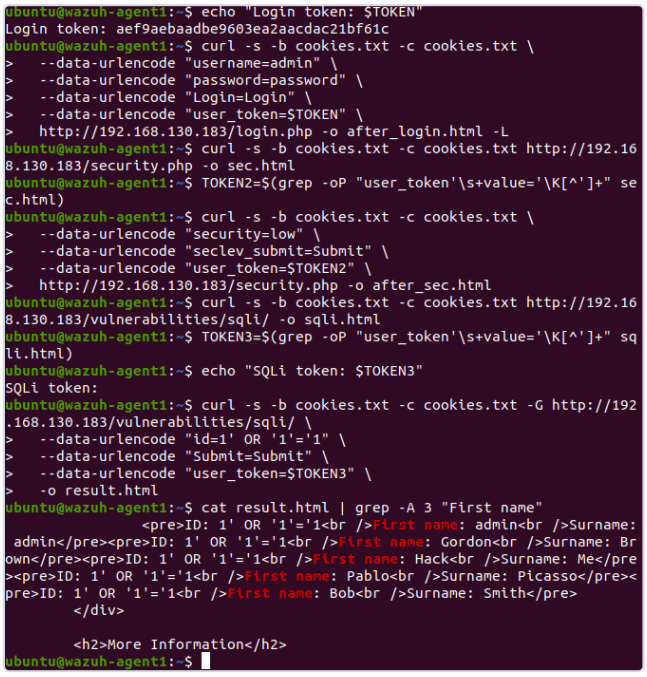



Figure 11 — Validation manuelle de l'injection SQL par récupération de résultats inattendus.

Après cette validation initiale, l'outil `sqlmap` a été utilisé pour confirmer automatiquement la vulnérabilité. Les captures mettent en évidence l'identification du point d'injection, la reconnaissance du système de gestion de base de données, l'énumération des tables et des colonnes, puis l'extraction du contenu de la table des utilisateurs. Cette séquence illustre la progression typique d'une exploitation automatisée d'une faiblesse SQL exploitable.




Figure 12 — Détection initiale du comportement injectable par *sqlmap*.

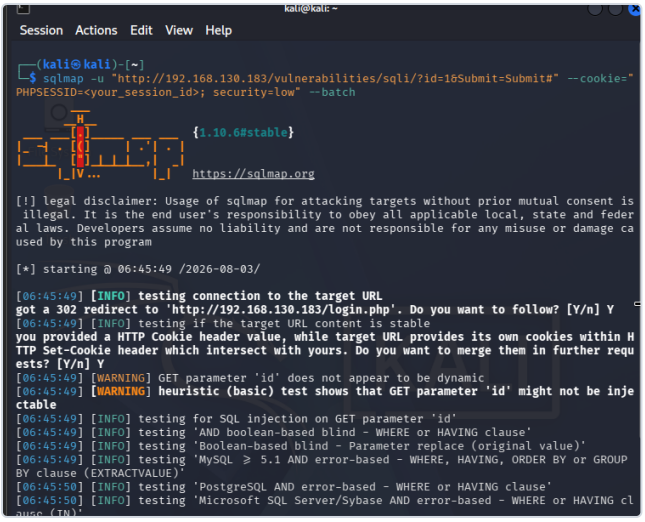



Figure 13 — Confirmation automatisée de la vulnérabilité en mode batch.

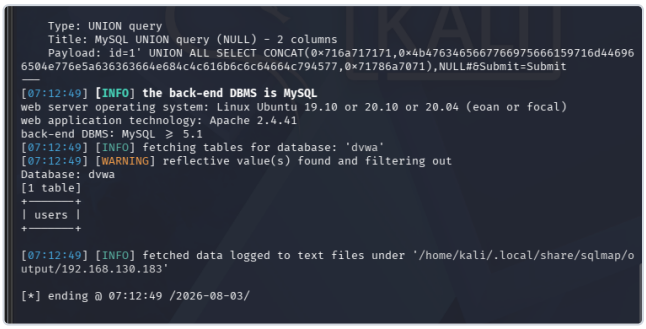



Figure 14 — Énumération des tables identifiées dans la base *dvwa*.




Figure 15 — Identification des colonnes de la table *users*.

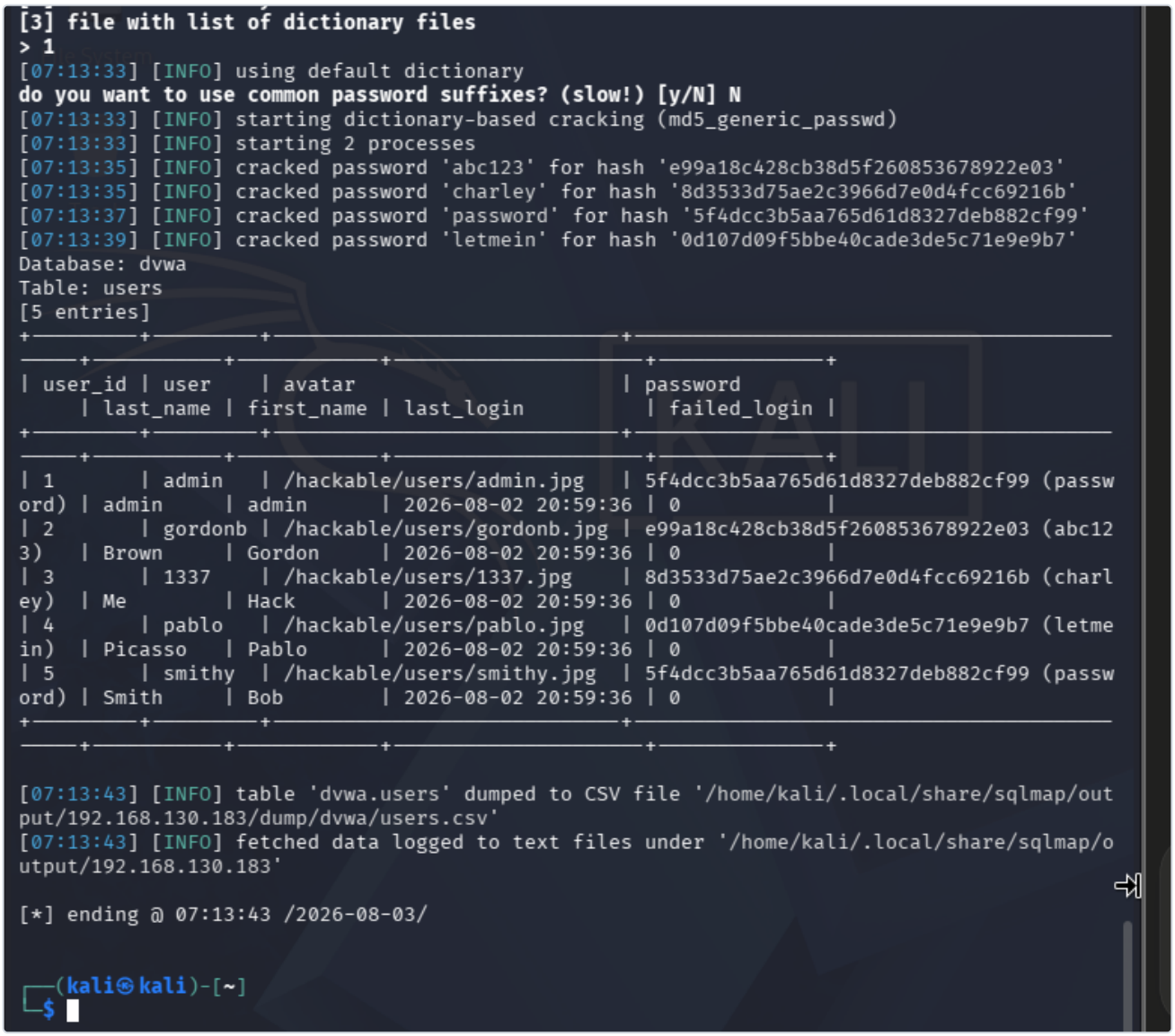



Figure 16 — Extraction du contenu de la table *users*, incluant les hachages et mots de passe retrouvés par dictionnaire.

## 7. Observation des événements et validation de la détection

La dernière étape de l'étude a consisté à vérifier que les actions offensives produisaient bien des événements visibles et interprétables dans l'interface du manager. Les captures suivantes montrent, d'une part, la présence d'alertes associées à des actions web considérées comme anormales et, d'autre part, le détail d'un document révélant l'URL sollicitée, l'adresse source, le type de journal mobilisé et la règle descriptive remontée par Wazuh.

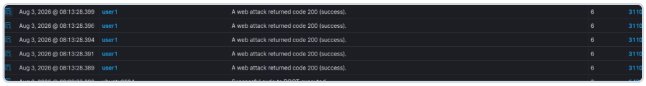


Figure 17 — Liste d’alertes relatives à des attaques web détectées avec succès.

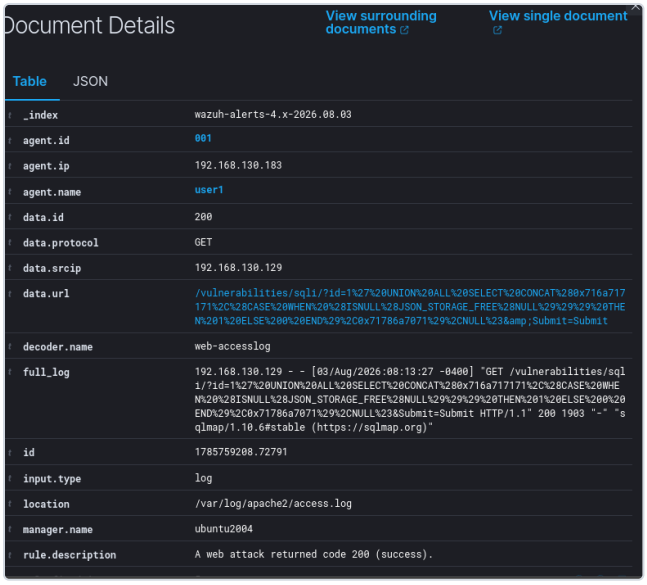



Figure 18 — Détail d’un événement incluant la charge utile de l’attaque et les métadonnées associées.

Les observations confirment que la chaîne de supervision est fonctionnelle. Les événements issus des accès au serveur, y compris les requêtes contenant des charges utiles offensives, sont bien collectés et indexés. La centralisation obtenue constitue une base pertinente pour approfondir l’analyse, créer des tableaux de bord thématiques ou ajuster des règles de corrélation plus spécifiques à certains scénarios d’attaque.

# 8. Discussion

D’un point de vue méthodologique, cette expérimentation présente un intérêt pédagogique important. Elle permet d’articuler trois dimensions souvent traitées séparément : le déploiement d’une cible applicative, l’exécution de scénarios offensifs contrôlés et l’observation défensive des traces produites. Le recours à DVWA simplifie la reproductibilité des tests, tandis que Wazuh fournit un cadre structurant pour l’analyse de la télémétrie générée.

Il convient toutefois de souligner que ce type de laboratoire ne reproduit pas l’intégralité de la complexité d’un environnement de production. Néanmoins, il constitue une étape particulièrement utile pour valider les principes fondamentaux de détection, comprendre le cheminement des événements depuis la source jusqu’au manager et préparer des travaux plus avancés sur la corrélation et la qualification d’attaques web.

# 9. Conclusion

En conclusion, le travail réalisé permet de valider de manière cohérente la chaîne complète de détection visée par le projet. L’application vulnérable a été correctement déployée sur la machine agent, la journalisation associée au service web a été rendue visible dans Wazuh, plusieurs scénarios offensifs ont été exécutés depuis Kali, et les événements correspondants ont pu être observés et interprétés au niveau du manager. Les résultats obtenus démontrent donc la pertinence de l’environnement construit pour l’étude et la démonstration de la détection d’attaques web.

Fin du rapport académique.